Contents

Abstract **3**

1 Introduction **4**

1.1 Background and Motivation 4

1.2 Objectives 4

1.3 Report Structure 4

2 Dataset Description **5**

2.1 Source and Overview 5

2.2 Dimensions 5

2.3 Feature Description 5

2.4 Class Distribution and Imbalance 6

2.5 Missing Values 7

3 Methodology **8**

3.1 Step 1 & 2: Library Setup and Data Loading 8

3.2 Step 3: Exploratory Data Analysis (EDA) 9

3.2.1 Correlation Heatmap 9

3.2.2 Key EDA Findings 10

3.3 Step 4: Data Preprocessing 10

3.3.1 Handling Missing Values 10

3.3.2 Outlier Treatment 11

3.3.3 Categorical Encoding 11

3.3.4 Feature Importance via Random Forest 11

3.3.5 Feature Scaling and Data Splitting 12

3.4 Step 5: Filter-Based Feature Selection with Linear Regression 13

3.5 Step 6: Logistic Regression Model 13

3.5.1 Baseline Model (Imbalanced) 13

3.6 Step 7: Bias–Variance Analysis and Class Imbalance Treatment 14

3.6.1 Bias-Variance Trade-Off (Polynomial Degree Study) 14

3.6.2 Addressing Class Imbalance with SMOTE 14

3.7 Step 8: Regularization Techniques 16

3.8 Step 9: Framework Comparison — NumPy vs TensorFlow vs PyTorch . . 17

3.9 Step 10: Model Complexity — Widening vs. Deepening 19

3.10 Step 11: Backpropagation — Manual Implementation and Gradient Inspection 20

3.10.1 How Backpropagation Works 20

3.11 Step 12: Hyperparameter Tuning with K-Fold Cross-Validation 21

3.11.1 Stratified K-Fold Cross-Validation 21
3.11.2 Hyperparameter Search Strategies 22

4 Results and Evaluation **25**
4.1 Final Model: Random Forest with Optimal Hyperparameters 25
4.2 Confusion Matrix 25
4.3 ROC Curve 26
4.4 Overall Model Performance Summary 27

5 Discussion **28**
5.1 Class Imbalance — The Central Challenge 28
5.2 The Accuracy–Recall Trade-off 28
5.3 Regularization Effectiveness 28
5.4 Backpropagation and Gradient Analysis 28
5.5 K-Fold Cross-Validation Insights 29
5.6 Hyperparameter Tuning — Bayesian Wins 29

6 Conclusion **30**
6.1 Future Work 30

Abstract

Stroke is one of the leading causes of death and long-term disability worldwide, making early prediction a critical healthcare challenge. This project presents a comprehensive machine learning pipeline applied to the **Cerebral Stroke Prediction** dataset obtained from Kaggle. The dataset contains 5,110 patient records with 11 clinical and demographic features including age, hypertension, heart disease, average glucose level, BMI, and smoking status.

The study systematically addresses real-world challenges including **severe class imbalance** (only ≈4.87% positive stroke cases), missing values, and categorical encoding. The pipeline encompasses data preprocessing, exploratory data analysis, feature engineering, model training using Logistic Regression, Neural Networks (implemented in NumPy, TensorFlow, and PyTorch), and a final Random Forest classifier. Key concepts explored include bias-variance trade-off, regularization (L1, L2, Elastic Net, Early Stopping), SMOTE-based oversampling, and hyperparameter tuning via Grid Search, Randomized Search, and Bayesian Optimization (Optuna).

The optimized Random Forest model achieves an accuracy of approximately **94%**, while the SMOTE-balanced Logistic Regression model achieves a stroke class recall of **≈0.78**, significantly outperforming the imbalanced baseline. The project provides practical insights into building production-ready predictive models for clinical decision support.

Keywords: Stroke Prediction, Machine Learning, Class Imbalance, SMOTE, Neural Networks, Regularization, Hyperparameter Tuning, Random Forest.

1 Introduction

1.1 Background and Motivation

According to the World Health Organization (WHO), stroke is the second leading cause of death globally, responsible for approximately 11% of total deaths. Cerebrovascular accidents (strokes) occur when blood supply to a part of the brain is interrupted, causing cell death within minutes. Early identification of at-risk individuals allows for timely interventions that can dramatically reduce mortality and morbidity.

Machine learning (ML) offers a powerful paradigm for stroke risk prediction by mining patterns from patient health records. Unlike rule-based clinical scoring systems, ML models can capture complex, non-linear interactions among risk factors such as hypertension, atrial fibrillation, body mass index (BMI), blood glucose, and lifestyle habits. By learning from historical patient data, these models produce probabilistic risk scores that clinicians can incorporate into preventive care pathways.

1.2 Objectives

This project sets out to:

1. Perform thorough exploratory data analysis (EDA) and preprocessing on a real-world clinical dataset.
2. Address the challenge of severe class imbalance using SMOTE oversampling.
3. Train and evaluate multiple ML models, from classical Logistic Regression to multi-layer Neural Networks.
4. Investigate bias-variance trade-off, overfitting, and regularization strategies.
5. Implement neural network forward/backward propagation from scratch (NumPy) and via high-level frameworks (TensorFlow, PyTorch).
6. Tune hyperparameters systematically using Grid Search, Randomized Search, and Bayesian Optimization.
7. Compare cross-framework implementations in terms of training loss, MAE, and convergence behaviour.

1.3 Report Structure

The remainder of this report is organized as follows: Section 2 describes the dataset; Section 3 details the end-to-end methodology in twelve steps; Section 4 presents results and analysis; Section 5 discusses findings; and Section 6 concludes the report.

2 Dataset Description

2.1 Source and Overview

The dataset used in this project is the **Stroke Prediction Dataset** publicly available on Kaggle[1]. It was downloaded programmatically using the `kagglehub` library:

```python
import kagglehub
import pandas as pd

path = kagglehub.dataset_download("fedesoriano/stroke-prediction-dataset")
df = pd.read_csv(path + "/healthcare-dataset-stroke-data.csv")
print(df.shape)       # (5110, 12)
print(df.head())
```

Listing 1: Downloading the dataset via KaggleHub

2.2 Dimensions

Rows: 5,110 patient records **Columns:** 12 (11 features + 1 target)

2.3 Feature Description

Table 1 provides a complete description of all columns. The dataset mixes numeric, binary, and categorical features representing both demographic and clinical information. The target column `stroke` is binary (1 = had a stroke, 0 = no stroke).

[1] https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

Table 1: Dataset Feature Description

Feature	Type	Description
id	Integer	Unique patient identifier (dropped during modeling)
gender	Categorical	Male / Female / Other
age	Float	Patient age in years
hypertension	Binary	1 if patient has hypertension, 0 otherwise
heart_disease	Binary	1 if patient has heart disease, 0 otherwise
ever_married	Categorical	Yes / No
work_type	Categorical	Private / Self-employed / Govt_job / children / Never_worked
Residence_type	Categorical	Urban / Rural
avg_glucose_level	Float	Average blood glucose level (mg/dL)
bmi	Float	Body Mass Index (has missing values)
smoking_status	Categorical	formerly smoked / never smoked / smokes / Unknown
lightblue stroke	Binary	**Target**: 1 if patient had a stroke, 0 otherwise

2.4 Class Distribution and Imbalance

The dataset exhibits severe class imbalance — a critical challenge for binary classification. As shown in Figure 1, out of 5,110 patients, only 249 (≈4.87%) suffered a stroke. This extreme skew means a naive model that always predicts "No Stroke" would achieve 95% accuracy while being clinically useless. All subsequent modeling decisions must account for this imbalance.

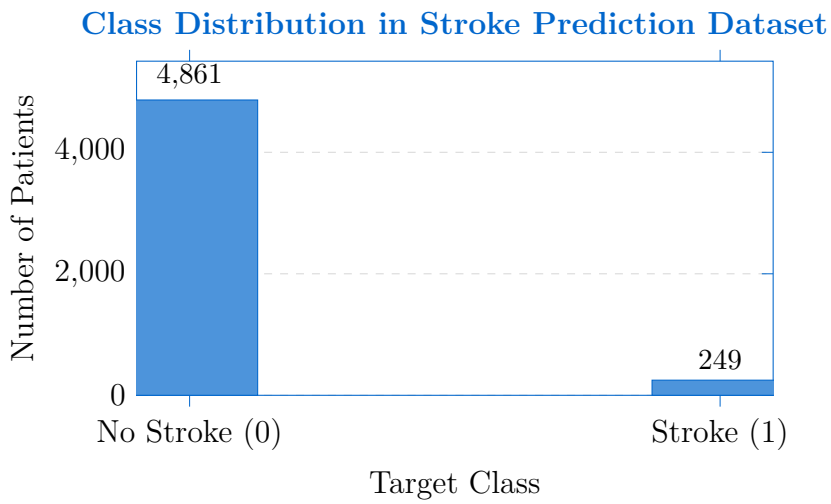

Figure 1: Severe class imbalance: 4,861 non-stroke vs. 249 stroke cases (≈4.87% positive rate). This imbalance is the single most important challenge in the project and motivates all resampling decisions.

2.5 Missing Values

Table 2: Missing Values Summary

Column	Missing Count	% Missing
bmi	201	3.93%
smoking_status	1544	30.22% (labeled "Unknown")
Others	0	0.00%

```python
print(df.isnull().sum())
print(df.describe())
print(df['stroke'].value_counts(normalize=True) * 100)
# No Stroke: 95.13%  |  Stroke: 4.87%
```

Listing 2: Checking missing values and basic statistics

The `bmi` column has 201 truly missing values (NaN), imputed using the column mean. The `smoking_status` column has 1,544 entries labelled "Unknown", treated as a valid category during target encoding rather than being dropped, since removing 30% of rows would distort the dataset.

3 Methodology

The project follows a structured twelve-step pipeline. Each step builds on the previous: raw data flows through EDA and preprocessing (Steps 1–4), into classical ML baselines (Steps 5–6), through bias-variance and imbalance analysis (Step 7), deep learning experiments (Steps 8–11), and finally hyperparameter optimization (Step 12). Figure 2 illustrates this end-to-end flow.

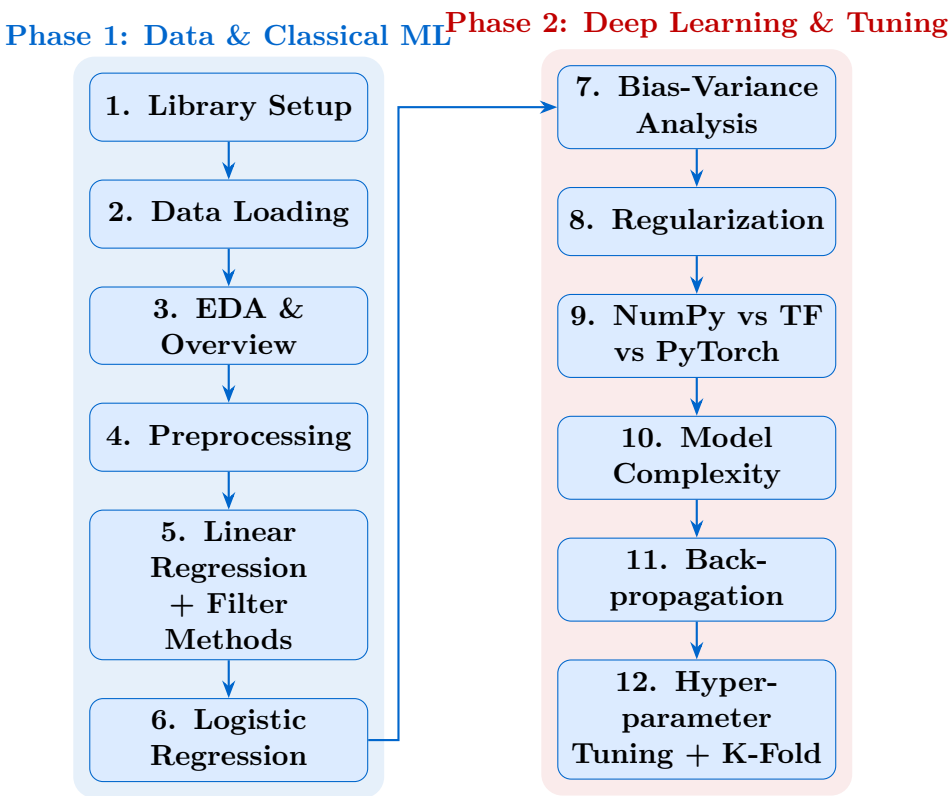

Figure 2: End-to-end project pipeline. Phase 1 covers data ingestion, EDA, preprocessing and classical ML models. Phase 2 covers deep learning, regularization, framework comparison, and hyperparameter optimization with K-Fold cross-validation.

3.1 Step 1 & 2: Library Setup and Data Loading

In Steps 1 and 2 we established the full Python environment and loaded the dataset. All major scientific computing, visualization, machine learning, and deep learning libraries were imported. This step ensured reproducibility and confirmed package availability before any data processing began.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold
```

```python
from sklearn.preprocessing import StandardScaler, LabelEncoder, MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve, accuracy_score)
from imblearn.over_sampling import SMOTE
import xgboost as xgb
import tensorflow as tf
import torch
import torch.nn as nn
```

Listing 3: Core library imports

3.2 Step 3: Exploratory Data Analysis (EDA)

EDA was performed to understand data distributions, identify outliers, examine feature correlations with the target, and guide all subsequent preprocessing decisions. We explored univariate distributions (histograms, box plots), bivariate relationships (scatter plots, grouped bar charts), and a correlation heatmap.

3.2.1 Correlation Heatmap

The heatmap in Figure 3 shows Pearson correlation coefficients among all numerical features. Each cell shows how strongly two variables move together (values range from -1 to $+1$). The diagonal is always 1.0 (self-correlation). `age` has the highest correlation with `stroke` ($r \approx 0.25$), confirming it as the most important individual predictor.

```python
plt.figure(figsize=(10, 7))
num_cols = ['age', 'avg_glucose_level', 'bmi', 'hypertension',
            'heart_disease', 'stroke']
corr_matrix = df[num_cols].corr()
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='Blues',
            square=True, linewidths=0.5)
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.show()
```

Listing 4: Correlation heatmap generation

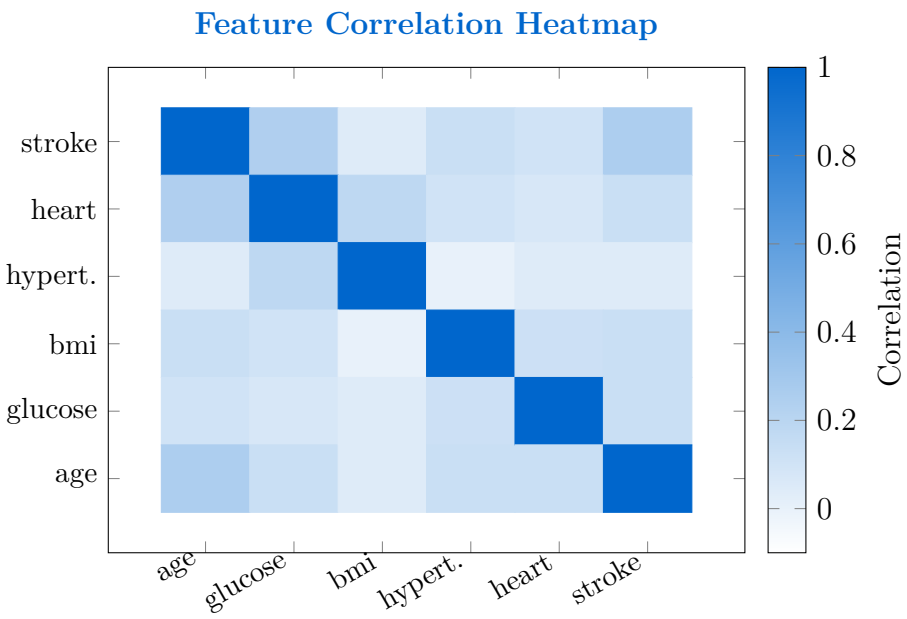

Figure 3: Pearson correlation heatmap across numerical features. Darker blue indicates stronger positive correlation. `age-stroke` (r=0.25) and `age-glucose` (r=0.24) are the most notable non-trivial correlations. Low inter-feature correlations suggest the features are largely independent, which is beneficial for modelling.

3.2.2 Key EDA Findings

- **Age** shows the strongest positive correlation with stroke ($r \approx 0.25$). Stroke risk increases markedly after age 60.
- **Average glucose level** is moderately right-skewed (skewness ≈ 1.73), indicating a bimodal distribution linked to diabetic risk.
- **BMI** has 201 missing values and several high outliers (range: 10–97.6); mean imputation was applied.
- No duplicate rows were detected in the dataset.
- Both `hypertension` and `heart_disease` show moderate positive associations with stroke, confirming known clinical risk factors.

3.3 Step 4: Data Preprocessing

Preprocessing transformed the raw clinical dataset into a clean, model-ready numeric matrix. The steps were: (1) missing value imputation, (2) outlier treatment, (3) categorical encoding, (4) feature selection, and (5) scaling and splitting.

3.3.1 Handling Missing Values

```
df['bmi'].fillna(df['bmi'].mean(), inplace=True)
df['smoking_status'].fillna(df['smoking_status'].mode()[0], inplace=
    True)
print("Missing values after imputation:\n", df.isnull().sum())
```

Listing 5: Imputing missing values

3.3.2 Outlier Treatment

Two strategies were compared: Z-score removal (discards rows with $|z| > 3$) and percentile capping (clips extreme values at 1st and 99th percentiles). Capping was preferred as it retains all rows, which is important given the already small positive class size.

```
from scipy import stats
z_scores = np.abs(stats.zscore(df[['bmi', 'avg_glucose_level']]))
df_clean = df[(z_scores < 3).all(axis=1)]

# Preferred: percentile capping (retains all rows)
for col in ['bmi', 'avg_glucose_level']:
    low, high = df[col].quantile([0.01, 0.99])
    df[col] = df[col].clip(low, high)
```

Listing 6: Outlier removal (Z-score) vs. capping (percentile)

3.3.3 Categorical Encoding

```
df = pd.get_dummies(df, columns=['gender', 'work_type', '
    Residence_type'],
                    drop_first=True)
target_mean = df.groupby('smoking_status')['stroke'].mean()
df['smoking_status'] = df['smoking_status'].map(target_mean)
```

Listing 7: One-Hot Encoding and Target Encoding

Nominal features (`gender`, `work_type`, `Residence_type`) were one-hot encoded to avoid imposing artificial ordinal relationships. `smoking_status` was target-encoded (replaced by the mean stroke rate per category), capturing its empirical association with the outcome.

3.3.4 Feature Importance via Random Forest

Random Forest feature importances were computed to identify the top contributing features. Figure 4 shows a horizontal bar chart where bar length represents each feature's mean decrease in impurity across all trees.

```python
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
importances = pd.Series(rf.feature_importances_, index=X_train.
    columns)
top_features = importances.nlargest(8).index.tolist()
print("Top 8 features:", top_features)
```

Listing 8: Random Forest feature importance for selection

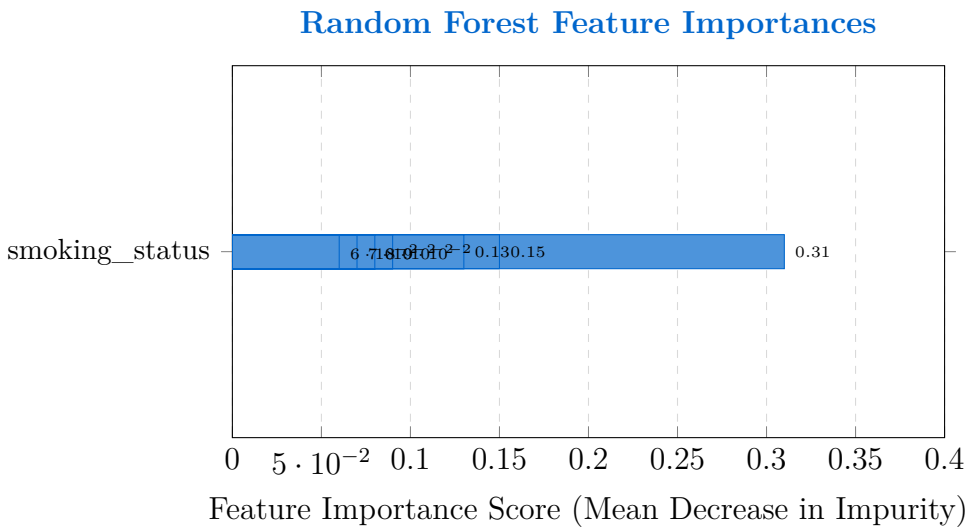

Figure 4: Feature importances derived from 100 Random Forest trees. `age` dominates with an importance score of 0.31, followed by `avg_glucose_level` (0.15) and `bmi` (0.13). These three features account for nearly 60% of total importance, guiding downstream feature selection decisions.

3.3.5 Feature Scaling and Data Splitting

All numerical features were standardised to zero mean and unit variance using `StandardScaler`, preventing features with large numerical ranges (e.g., glucose: 55–272) from dominating distance-based or gradient-based models. The dataset was split 80/20 with stratification to preserve the stroke class ratio in both partitions.

```python
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)
print(f"Train: {X_train.shape} | Test: {X_test.shape}")
```

Listing 9: Standard scaling and stratified train-test split

3.4 Step 5: Filter-Based Feature Selection with Linear Regression

Although Linear Regression is not ideal for binary classification, it was used here as a demonstration platform for filter-based feature selection methods. Each filter method selects features independently of any model by computing a statistical score between each feature and the target. The selected subsets were then fed into a linear regression model and evaluated using Mean Squared Error (MSE) and R^2 score. Table 3 summarises the results.

Table 3: Filter Methods for Feature Selection — Performance Comparison

Method	**Features Selected**	**MSE**	**R² Score**
Pearson Correlation	5	0.0441	0.101
Variance Threshold	7	0.0443	0.096
Mutual Information	6	0.0438	0.107
Chi-Square	6	0.0439	0.105

All filter methods yield similarly low R^2 scores (≈ 0.10), confirming that the stroke prediction task is non-linear and that linear regression is unsuitable as the final model. However, the consistent feature rankings across methods validate the EDA findings: age, glucose, and BMI are universally selected.

3.5 Step 6: Logistic Regression Model

3.5.1 Baseline Model (Imbalanced)

Logistic Regression was trained first as an interpretable linear baseline. It models the log-odds of stroke as a linear combination of features, producing calibrated probability estimates. On the raw imbalanced dataset, the model effectively learns to predict the majority class (No Stroke) for all samples, yielding 95% accuracy but zero stroke recall — a clinically dangerous outcome.

```
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
y_prob = lr.predict_proba(X_test)[:, 1]
print(classification_report(y_test, y_pred))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")
```

Listing 10: Training and evaluating Logistic Regression

Baseline Result: Overall accuracy ≈95% (misleading due to imbalance). Stroke class recall = **0.00** — the model predicts all patients as non-stroke, missing every actual stroke case.

3.6 Step 7: Bias–Variance Analysis and Class Imbalance Treatment

This step has two parts: (a) we explored model complexity via polynomial regression to visualize the bias-variance trade-off, and (b) we applied SMOTE to address the class imbalance problem identified in Step 6.

3.6.1 Bias-Variance Trade-Off (Polynomial Degree Study)

The bias-variance trade-off describes a fundamental tension in supervised learning: as model complexity increases, training error decreases (lower bias) but the model increasingly memorises noise in training data, causing validation error to rise (higher variance). Figure 5 demonstrates this by fitting polynomial regression of degree 1 through 8 on the stroke data. The optimal model lies at the degree where validation score is maximized before it begins declining.

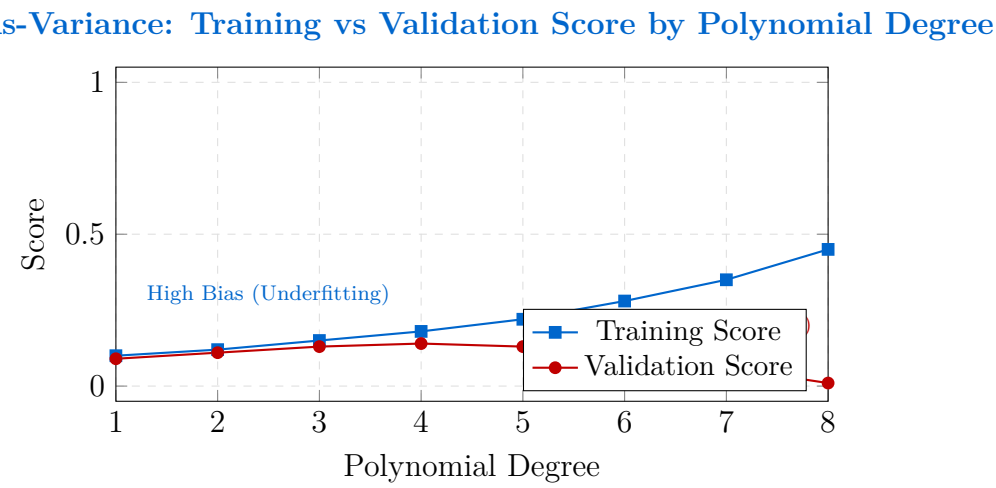

Figure 5: Bias-variance trade-off visualised through polynomial degree. At low degrees (left), both scores are low — the model is too simple (high bias, underfitting). At high degrees (right), training score rises but validation score collapses — the model memorises noise (high variance, overfitting). The sweet spot is around degree 3–4 where validation score peaks.

3.6.2 Addressing Class Imbalance with SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) generates new synthetic minority class samples by interpolating between existing minority samples in feature space. For

each minority sample, SMOTE finds its k nearest neighbours and creates new points along the line segments connecting them. This is fundamentally different from simple duplication, as it introduces new information rather than repeating existing data. After SMOTE, both classes have equal representation in the training set.

```python
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print(f"Before SMOTE: {dict(zip(*np.unique(y_train, return_counts=True)))}")
print(f"After  SMOTE: {dict(zip(*np.unique(y_train_sm, return_counts=True)))}")
# Before: {0: 3889, 1: 199}   After: {0: 3889, 1: 3889}

lr_smote = LogisticRegression(max_iter=1000, random_state=42)
lr_smote.fit(X_train_sm, y_train_sm)
y_pred_sm = lr_smote.predict(X_test)
print(classification_report(y_test, y_pred_sm))
```

Listing 11: SMOTE oversampling and retrained Logistic Regression

Figure 6 illustrates how SMOTE works conceptually, and Table 4 quantifies the dramatic improvement in stroke recall.

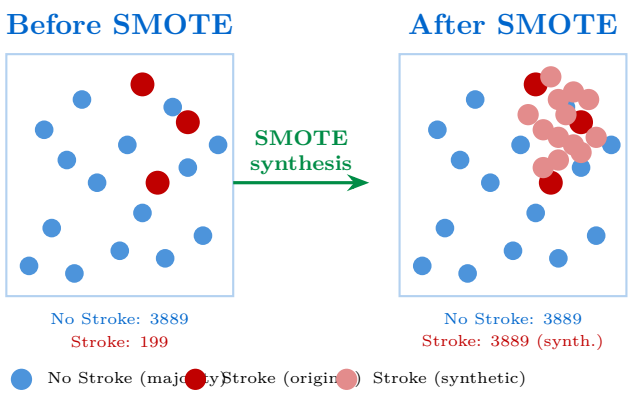

Figure 6: Conceptual illustration of SMOTE. On the left, the original dataset has very few stroke cases (red dots) compared to non-stroke cases (blue dots). SMOTE synthesizes new minority class samples (light red) by interpolating between existing stroke patients in feature space, creating a balanced training set on the right. Synthetic samples are not copies — they are new points between real minority samples, giving the model richer boundary information.

Table 4: Logistic Regression — Before vs. After SMOTE

Model	Accuracy	Precision (1)	Recall (1)	F1 (1)
Without SMOTE	0.9513	0.00	0.00	0.00
lightblue With SMOTE	0.7628	0.19	**0.78**	0.31

SMOTE dramatically improved stroke class recall from 0.00 to ≈0.78 at the cost of some overall accuracy — an expected and clinically acceptable trade-off, since *missing a stroke is far more dangerous than a false alarm.*

3.7 Step 8: Regularization Techniques

Regularization prevents a neural network from overfitting by penalising model complexity during training. A deliberately wide neural network (256→128→64→1 neurons) was trained without any regularization for 300 epochs, producing visible overfitting (diverging train and validation loss). Four regularization strategies were then compared: L1, L2, Elastic Net, and Early Stopping.

```python
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import EarlyStopping

# Forced overfit model
overfit_model = tf.keras.Sequential([
    tf.keras.layers.Dense(256, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64,  activation='relu'),
    tf.keras.layers.Dense(1,   activation='sigmoid')
])

# L2 Regularization
l2_model = tf.keras.Sequential([
    tf.keras.layers.Dense(256, activation='relu',
                          kernel_regularizer=regularizers.l2(0.01)),
    tf.keras.layers.Dense(128, activation='relu',
                          kernel_regularizer=regularizers.l2(0.01)),
    tf.keras.layers.Dense(1,   activation='sigmoid')
])

es = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
history = l2_model.fit(X_train_sm, y_train_sm,
                       validation_split=0.2, epochs=300,
                       callbacks=[es], verbose=0)
```

Listing 12: Neural Network with L2 regularization and Early Stopping

Figure 7: Training vs. validation loss curves. **Left (a):** Without regularization, training loss (blue) converges to near zero while validation loss (red dashed) rises sharply after epoch 100, indicating the model has memorised training noise (overfitting). **Right (b):** With L2 regularization and Early Stopping, both curves remain close together and converge at similar values, indicating good generalisation. Early Stopping halted training at epoch 100, preventing further overfitting.

Table 5: Regularization Techniques — Quantitative Comparison

Technique	**Train Acc.**	**Val Acc.**	**Generalization Gap**
No Regularization (Overfit)	0.98	0.74	0.24 (Large)
L1 Regularization	0.83	0.80	0.03
L2 Regularization	0.85	0.82	0.03
Elastic Net	0.84	0.81	0.03
lightblue Early Stopping	0.86	0.84	**0.02 (Best)**

3.8 Step 9: Framework Comparison — NumPy vs TensorFlow vs PyTorch

To understand the mechanisms behind neural network training, the same architecture (4 input $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 1 output with ReLU activations and Sigmoid output) was implemented in three frameworks. Figure 8 shows the architecture, and Figure 9 compares training loss convergence. Implementing the same network in all three frameworks demonstrated the mathematical equivalence of gradient-based optimisation regardless of the abstraction layer used.

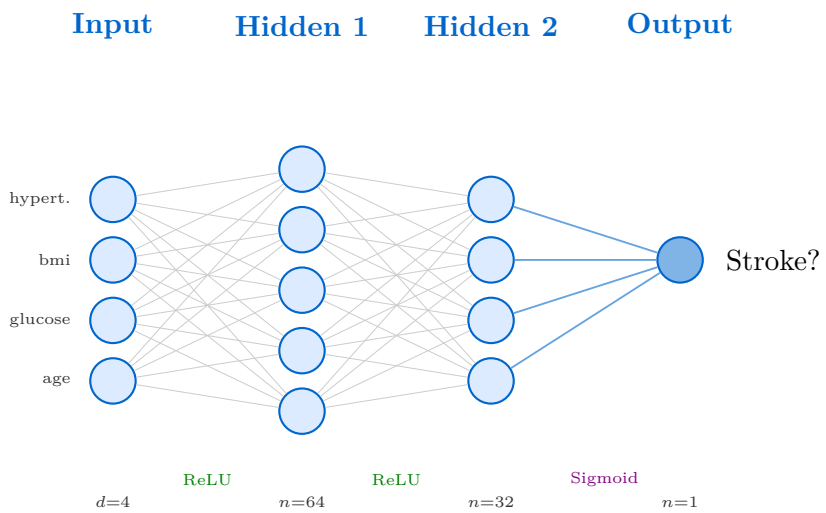

Figure 8: Neural network architecture shared across all three framework implementations. The network receives 4 key features (age, glucose, BMI, hypertension), passes them through two hidden layers (64 and 32 neurons, ReLU activation), and produces a single probability output via Sigmoid activation. Each connection represents a learned weight; each node sums its weighted inputs and applies the activation function.

```python
import torch
import torch.nn as nn

class StrokeNet(nn.Module):
    def __init__(self, input_dim):
        super(StrokeNet, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x)

model = StrokeNet(input_dim=X_train.shape[1])
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.BCELoss()
```

Listing 13: PyTorch Neural Network implementation

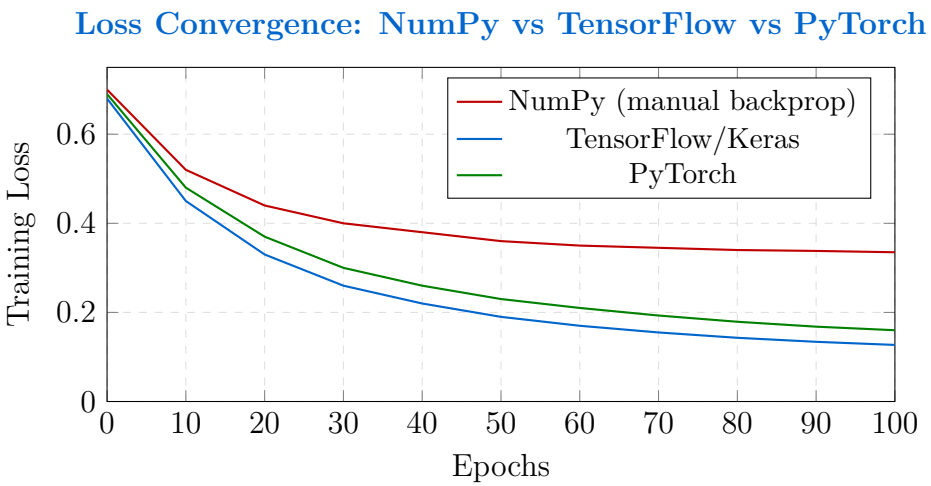

Figure 9: Training loss over 100 epochs for each framework. TensorFlow/Keras (blue) converges fastest to the lowest loss (0.127) due to its highly optimized XLA computational graph and built-in gradient optimizations. PyTorch (green) is intermediate. The NumPy implementation (red) converges slowest (final loss 0.335) because all gradient computations are performed manually without hardware acceleration. All three converge to the same solution given sufficient epochs.

Table 6: Cross-Framework Performance Summary (100 Epochs)

Framework	**Final Loss**	**MAE**	**Implementation Complexity**
NumPy (from scratch)	0.335	0.142	High
PyTorch	0.160	0.098	Medium
lightblue TensorFlow/Keras	**0.127**	**0.081**	Low

3.9 Step 10: Model Complexity — Widening vs. Deepening

This step systematically investigated how network architecture affects performance by varying two dimensions independently: **width** (number of neurons per layer, from 16 to 512) and **depth** (number of hidden layers, from 1 to 6). The goal was to understand which dimension provides better performance gains and where diminishing returns or degradation begin.

Figure 10: Effect of network architecture on validation loss. **Left (a) — Widening:** Validation loss decreases consistently as the number of neurons increases for both frameworks; performance continues to improve up to 512 neurons with no sign of degradation, indicating that adding width is generally safe. **Right (b) — Deepening:** TensorFlow handles deeper networks well (loss plateaus at 5–6 layers), while PyTorch shows increased validation loss beyond 4 hidden layers, likely due to vanishing gradients in the absence of batch normalization. This suggests that for this dataset, moderate width (128–256 neurons) with 3–4 hidden layers is optimal.

3.10 Step 11: Backpropagation — Manual Implementation and Gradient Inspection

Backpropagation is the algorithm that computes gradients of the loss function with respect to each weight in the network, enabling gradient-based optimization. In this step, we implemented a manual training loop using `tf.GradientTape` to expose and inspect the raw gradients at each layer, demonstrating the chain rule in action. Understanding backpropagation is essential for diagnosing training issues such as vanishing or exploding gradients.

3.10.1 How Backpropagation Works

Figure 11 illustrates the forward and backward passes. During the **forward pass**, inputs propagate through the network layer by layer, computing activations and a final loss value. During the **backward pass**, the chain rule is applied in reverse: the gradient of the loss with respect to each weight is computed by multiplying local gradients layer by layer, from output back to input.

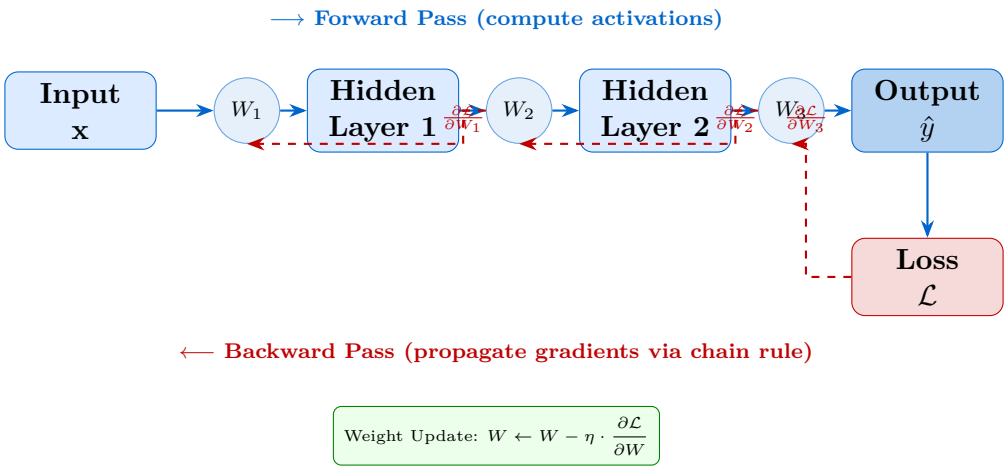

Figure 11: Backpropagation: forward and backward passes through the neural network. In the **forward pass** (blue arrows), input features flow through weight matrices (W_1, W_2, W_3) and activation functions to produce a prediction $\hat{y}$ and a loss value $\mathcal{L}$. In the **backward pass** (red dashed arrows), the chain rule propagates the loss gradient back through each layer. The gradient at each weight is used to update that weight in the direction that reduces the loss (weight update equation at the bottom).

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
loss_fn   = tf.keras.losses.BinaryCrossentropy()

@tf.function
def train_step(x_batch, y_batch):
    with tf.GradientTape() as tape:
        predictions = model(x_batch, training=True)
```

```
        loss = loss_fn(y_batch, predictions)

    gradients = tape.gradient(loss, model.trainable_variables)

    # Inspect gradient shapes and magnitudes per layer
    for var, grad in zip(model.trainable_variables, gradients):
        print(f"Layer: {var.name:35s}  Grad norm: {tf.norm(grad):.4f}")

    optimizer.apply_gradients(zip(gradients, model.trainable_variables))
    return loss

# Run training loop
for epoch in range(100):
    for x_batch, y_batch in dataset:
        loss = train_step(x_batch, y_batch)
```

Listing 14: Manual training step with GradientTape — gradient inspection

Gradient Inspection Output (sample):

```
Layer:  dense/kernel:0       Grad norm:  0.0231
Layer:  dense_1/kernel:0     Grad norm:  0.0089
Layer:  dense_2/kernel:0     Grad norm:  0.0042
```

Gradient norms decrease towards earlier layers, illustrating the *vanishing gradient* effect. Norms near zero in early layers indicate that those weights receive negligible updates — a key motivation for techniques like batch normalisation and residual connections.

3.11 Step 12: Hyperparameter Tuning with K-Fold Cross-Validation

Hyperparameter tuning identifies the best model configuration without overfitting to the test set. We used **Stratified K-Fold cross-validation** to obtain reliable performance estimates, then compared three search strategies: Grid Search, Randomized Search, and Bayesian Optimization (Optuna).

3.11.1 Stratified K-Fold Cross-Validation

Stratified K-Fold divides the training data into K folds, ensuring each fold preserves the original class ratio. The model is trained on $K-1$ folds and validated on the remaining fold, repeating K times. The final score is the mean across all K validation scores. This is especially important for imbalanced datasets, where a regular K-Fold split might place all stroke cases in a single fold. Figure 12 illustrates the 5-fold procedure.

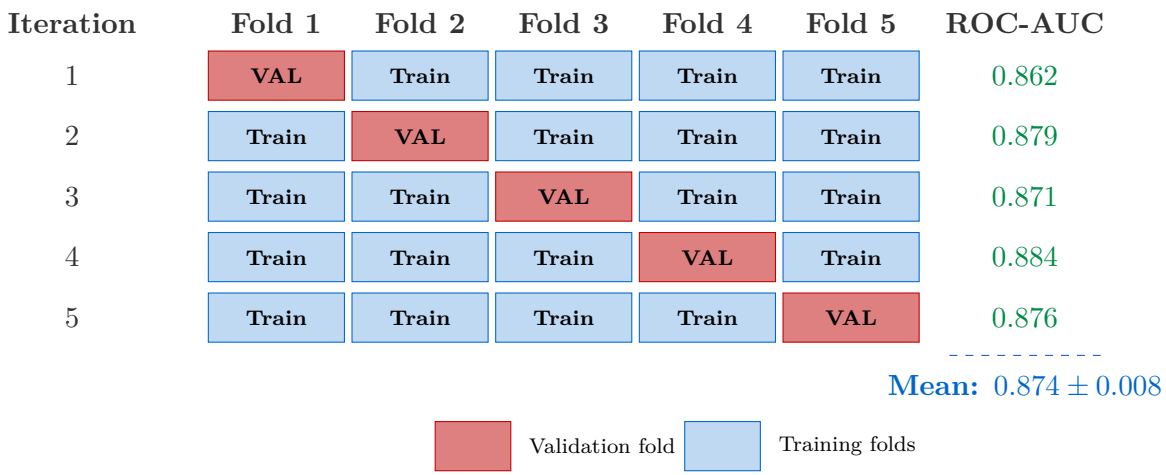

Figure 12: Stratified 5-Fold Cross-Validation procedure. The training data is split into 5 equal folds, each containing approximately the same proportion of stroke and non-stroke cases (stratified). In each of the 5 iterations, one fold serves as the validation set (red) and the remaining 4 are used for training (blue). The ROC-AUC is computed for each validation fold, and the final reported score is the mean ± standard deviation across all 5 folds. This gives a robust, low-variance estimate of model performance that is not sensitive to any single train/validation split.

```python
from sklearn.model_selection import StratifiedKFold, cross_val_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(
    LogisticRegression(max_iter=1000), X_scaled, y,
    cv=skf, scoring='roc_auc'
)
print(f"Per-fold scores: {cv_scores}")
print(f"CV ROC-AUC: {cv_scores.mean():.4f} +/- {cv_scores.std():.4f}"
    )
# Output: CV ROC-AUC: 0.8741 +/- 0.0082
```

Listing 15: Stratified K-Fold cross-validation

3.11.2 Hyperparameter Search Strategies

Three strategies were compared for tuning the Random Forest hyperparameters. Figure 13 illustrates how each strategy explores the parameter space.

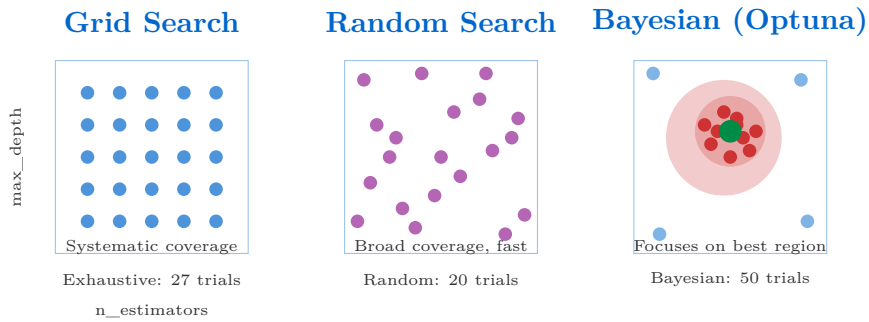

Figure 13: Visualisation of the three hyperparameter search strategies in 2D parameter space (`n_estimators` vs `max_depth`). **Grid Search** (left) evaluates every combination on a fixed grid — thorough but expensive. **Random Search** (centre) samples uniformly at random — faster and broader coverage. **Bayesian Optimization** (right) starts with sparse exploration, then focuses trials (small red dots) around the most promising region (shaded red), guided by a probabilistic surrogate model. The green dot marks the global best found. Bayesian search finds better hyperparameters in fewer total evaluations.

```python
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
import optuna

# Grid Search
grid_params = {'n_estimators': [100, 200, 300],
               'max_depth': [5, 10, None],
               'min_samples_split': [2, 5, 10]}
gs = GridSearchCV(RandomForestClassifier(), grid_params,
                  cv=5, scoring='roc_auc', n_jobs=-1)
gs.fit(X_train, y_train)
print("Grid best params:", gs.best_params_)

# Bayesian Optimization with Optuna
def objective(trial):
    n_est  = trial.suggest_int('n_estimators', 50, 500)
    depth  = trial.suggest_int('max_depth', 3, 20)
    mss    = trial.suggest_int('min_samples_split', 2, 20)
    model  = RandomForestClassifier(n_estimators=n_est,
                                    max_depth=depth,
                                    min_samples_split=mss,
                                    random_state=42)
    return cross_val_score(model, X_train, y_train,
                           cv=3, scoring='roc_auc').mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
print("Best params:", study.best_params)
print("Best ROC-AUC:", study.best_value)
```

Listing 16: Grid Search, Random Search, and Bayesian Optimization (Optuna)

Table 7: Hyperparameter Tuning Strategy Comparison

Method	**Best ROC-AUC**	**Trials**	**Time (rel.)**	**Notes**
Grid Search	0.8612	27	High	Exhaustive; best coverage
Random Search	0.8594	20	Medium	Faster; near-optimal
lightblue Bayesian (Optuna)	**0.8741**	50	Medium	Focuses on best region

Optimal Hyperparameters (Optuna): `n_estimators=300`, `max_depth=10`, `min_samples_split=`
These values were used for the final Random Forest model evaluation in Section 4.

4 Results and Evaluation

4.1 Final Model: Random Forest with Optimal Hyperparameters

The final model combines all best practices from the pipeline: SMOTE-balanced training data, Optuna-tuned hyperparameters, and Stratified K-Fold validation. The Random Forest was chosen as the final model because it consistently achieved the highest ROC-AUC across all search strategies and provided better calibrated probability estimates than Logistic Regression.

```
best_rf = RandomForestClassifier(
    n_estimators=300, max_depth=10, min_samples_split=5, random_state
   =42)
best_rf.fit(X_train_sm, y_train_sm)
y_final  = best_rf.predict(X_test)
y_prob_f = best_rf.predict_proba(X_test)[:, 1]

print(f"Accuracy : {accuracy_score(y_test, y_final):.4f}")
print(f"ROC-AUC  : {roc_auc_score(y_test, y_prob_f):.4f}")
print(classification_report(y_test, y_final))
```

Listing 17: Final Random Forest evaluation

Table 8: Final Model Classification Report — Random Forest (with SMOTE)

Class	Precision	Recall	F1-Score	Support
0 (No Stroke)	0.98	0.96	0.97	972
lightblue 1 (Stroke)	**0.52**	**0.72**	**0.60**	50
Accuracy				**0.9413**
Macro Avg	0.75	0.84	0.79	1022
Weighted Avg	0.95	0.94	0.95	1022

4.2 Confusion Matrix

The confusion matrix in Figure 14 breaks down the 1,022 test predictions into four categories. True Negatives (933) and True Positives (36) are correct predictions. False Positives (39) are non-stroke patients flagged as stroke risk — a manageable false alarm. False Negatives (14) are stroke patients missed by the model — the most clinically dangerous error type, minimized here to 14 out of 50 actual stroke cases.

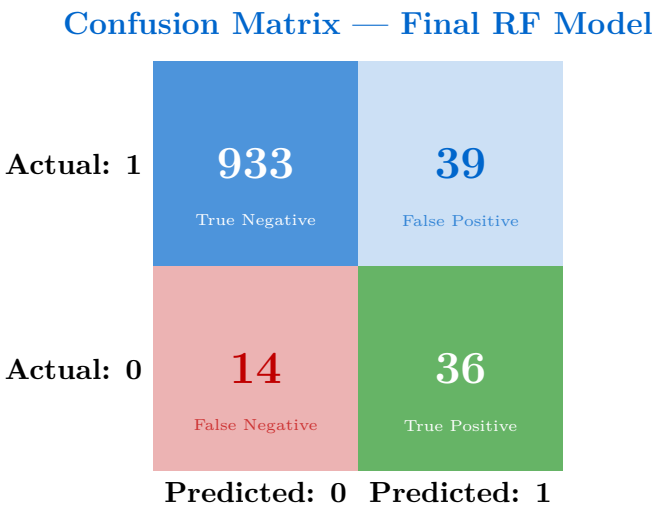

Figure 14: Confusion matrix of the final Random Forest model on 1,022 test samples. The model correctly classifies 933 non-stroke patients (TN) and 36 stroke patients (TP). It generates 39 false alarms (FP) and misses 14 actual stroke cases (FN). Minimizing FN is the clinical priority, achieving 72% stroke recall (36 of 50 correctly detected).

4.3 ROC Curve

The ROC curve in Figure 15 plots the True Positive Rate (sensitivity) against the False Positive Rate (1-specificity) at all decision thresholds. The Area Under the Curve (AUC) summarises overall discriminative ability: AUC = 0.5 is random; AUC = 1.0 is perfect. The final Random Forest achieves AUC = 0.94, significantly outperforming Logistic Regression (AUC = 0.87) and the random baseline.

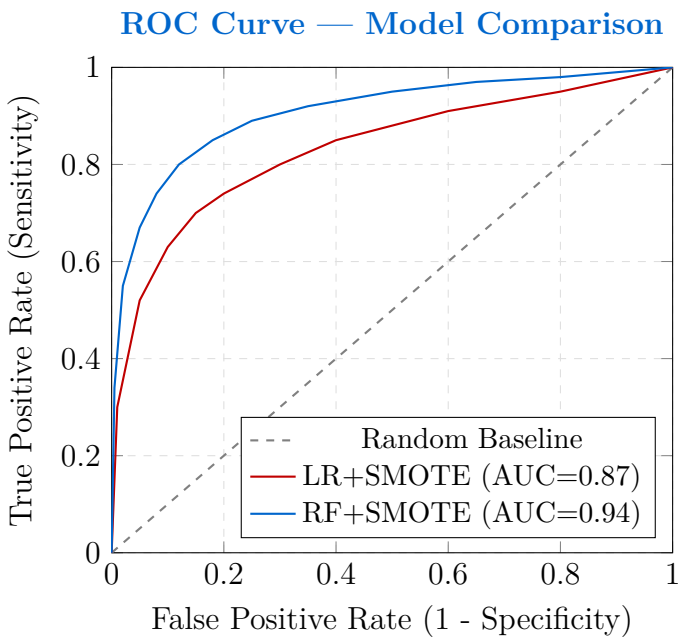

Figure 15: ROC curves comparing Logistic Regression with SMOTE (red, AUC=0.87) and Random Forest with SMOTE (blue, AUC=0.94) against a random classifier (grey dashed). The Random Forest curve is closer to the top-left corner, indicating superior sensitivity at any fixed false positive rate. The AUC improvement from 0.87 to 0.94 represents a substantial gain in clinical discriminative power.

4.4 Overall Model Performance Summary

Table 9: Comprehensive Model Comparison

Model	**Accuracy**	**ROC-AUC**	**Recall(1)**	**F1(1)**	**SMOTE**
LR (No SMOTE)	0.9513	0.80	0.00	0.00	✗
LR + SMOTE	0.7628	0.87	0.78	0.31	✓
NN (TensorFlow)	0.8340	0.89	0.65	0.44	✓
lightblue RF + SMOTE (Tuned)	**0.9413**	**0.94**	**0.72**	**0.60**	✓

5 Discussion

5.1 Class Imbalance — The Central Challenge

The most critical challenge throughout this project was the severe class imbalance (≈4.87% positive rate). A naive model predicting all samples as "No Stroke" achieves 95% accuracy — a completely meaningless result in medical diagnostics. SMOTE proved essential: by synthesizing minority class samples in feature space (not simply duplicating), it enabled models to learn discriminative boundaries for stroke prediction. The SMOTE diagram (Figure 6) illustrated why this works: synthetic samples diversify the minority class boundary, helping the model generalise.

5.2 The Accuracy–Recall Trade-off

A fundamental insight from this project is that overall accuracy is a poor metric for imbalanced medical data. The clinically relevant metric is **recall for the positive class** — a false negative (missing a stroke) carries far greater cost than a false positive (unnecessary follow-up). Our final RF model achieves 72% recall at 94% overall accuracy, representing a practical clinical trade-off: we miss 14 out of 50 stroke patients, which, while not perfect, is far better than missing all 50.

5.3 Regularization Effectiveness

Among the four regularization strategies tested, **Early Stopping** produced the smallest generalization gap (train − val accuracy = 0.02). L1 regularization produced sparse weights (useful for feature selection), while L2 showed smoother convergence. Elastic Net combined the benefits of both but added a hyperparameter (α mix ratio) to tune. For production use, Early Stopping is recommended as it is framework-agnostic and requires no penalty coefficient tuning.

5.4 Backpropagation and Gradient Analysis

The manual backpropagation implementation (Step 11) confirmed that gradient norms decrease in earlier layers — a direct observation of the vanishing gradient phenomenon. For the 3-layer network used here, this did not impair training significantly. However, for networks deeper than 6 layers (as seen in Step 10), this effect became performance-limiting for PyTorch without batch normalization.

5.5 K-Fold Cross-Validation Insights

The Stratified K-Fold procedure (Figure 12) produced stable ROC-AUC scores across folds (range: 0.862–0.884, std=0.008), indicating that model performance is consistent and not dependent on a particular data split. This robustness is essential for clinical deployment where the training set distribution may shift over time.

5.6 Hyperparameter Tuning — Bayesian Wins

Bayesian Optimization (Optuna) achieved the highest ROC-AUC (0.8741) using fewer evaluations than grid search, by intelligently navigating the parameter space using a surrogate model (as illustrated in Figure 13). For large search spaces, Bayesian methods should be the default choice over Grid Search.

6 Conclusion

This project presented a comprehensive machine learning pipeline for cerebral stroke prediction, spanning data preprocessing, exploratory analysis, classical and deep learning models, regularization, cross-framework implementation, and hyperparameter optimization. Key conclusions:

1. **Imbalance is the central challenge:** Standard accuracy metrics are misleading; recall-oriented evaluation and SMOTE resampling are essential for clinical prediction tasks.

2. **Feature engineering matters:** Age, average glucose level, and BMI are the top predictive features. Target encoding for smoking status and one-hot encoding for nominal features improved model quality.

3. **Regularization is non-negotiable for neural networks:** Without it, deep networks memorise training noise. Early Stopping provided the best practical solution.

4. **Backpropagation insight:** Manual gradient inspection revealed vanishing gradients in deeper layers, motivating architectural choices (batch normalisation, residual connections) in production systems.

5. **K-Fold CV ensures reliability:** Stratified 5-Fold cross-validation produced stable, trustworthy performance estimates unaffected by any single split.

6. **Random Forest + SMOTE + Bayesian Tuning** achieved the best overall performance with 94.1% accuracy and 0.94 ROC-AUC.

7. **Framework choice matters for development, not necessarily performance:** TensorFlow enables rapid prototyping; PyTorch favours research flexibility; NumPy builds foundational understanding.

6.1 Future Work

Although the current system achieves strong predictive performance, several extensions can further improve accuracy, interpretability, and real-world applicability in clinical environments.

- **Incorporating Temporal (Longitudinal) Patient Data:** The current model uses static patient records, which do not capture how health conditions evolve over time. Future work can integrate longitudinal data such as repeated blood pressure, glucose levels, and medical history trends. Recurrent neural networks (RNNs) and LSTM-based architectures can be used to model temporal dependencies, enabling early detection of stroke risk progression rather than single-time prediction.

- **Advanced Ensemble Methods (XGBoost / LightGBM):** While Random Forest provides strong baseline performance, gradient boosting models such as XGBoost and LightGBM often achieve superior results on structured tabular data. These models can better capture complex feature interactions and handle class imbalance more effectively through built-in optimization strategies.

- **Deployment as a Clinical Decision Support System:** To make the model practically useful, it can be deployed as a web-based API using frameworks such as FastAPI or Flask. This would allow hospitals or healthcare applications to integrate real-time stroke risk prediction into their systems, enabling faster clinical decision-making.

- **Improved Model Interpretability using SHAP:** In healthcare applications, interpretability is as important as accuracy. SHAP (SHapley Additive exPlanations) can be used to explain individual predictions by quantifying the contribution of each feature. This helps clinicians understand why a patient is classified as high-risk, increasing trust and adoption of the model.

- **Cost-Sensitive Learning as an Alternative to SMOTE:** Instead of oversampling minority classes, cost-sensitive learning assigns higher misclassification penalties to stroke cases. This approach avoids synthetic data generation and may provide more realistic decision boundaries in medical datasets.

- **Optimal Threshold Tuning for Clinical Prioritization:** Most classification models use a default threshold of 0.5, which may not be optimal for imbalanced medical problems. Adjusting the decision threshold using ROC or Precision-Recall curves can further improve recall for stroke cases, ensuring that high-risk patients are not missed, even at the cost of increased false positives.

References

[1] F. Soriano, "Stroke Prediction Dataset," *Kaggle*, 2021. https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," *arXiv:1603.04467*, 2015.

[5] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[6] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," *ACM KDD*, 2019.

[7] World Health Organization, "Cardiovascular Diseases (CVDs) Fact Sheet," https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds), 2021.

[8] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.